

## 🔍 Оценка выполнения второго спринта

---

### ☑ 1. Разделение монолита и подготовка микросервисов

#### Выполнено:

- Созданы директории `auth_service`, `posts_service`, `subscription_service`.
  - У каждого сервиса своя БД (в docker-compose), `Dockerfile`, `requirements.txt`, `.env.local`, `config.py`.
  - Применены `pydantic-settings`, конфигурация вынесена корректно.
  - `docker-compose.yml` запускает все сервисы, включая RabbitMQ и PostgreSQL.
- 

### ☑ 2. Auth Service (регистрация, JWT)

#### Выполнено:

- Регистрация реализована (`/register`), используется `bcrypt` и JWT.
- `users.py` обрабатывает создание пользователей.
- `security.py` реализует `get_current_user`, `verify_password`, `create_access_token`.

#### 🕒 Недостатки:

- Нет эндпоинта `/login`.
- Не реализовано подтверждение email и восстановление пароля.
- Используется `mix sync/async` — в `auth.py` подключение sync-сессии.

#### 💡 Рекомендации:

- Добавить `/login` с генерацией JWT.
  - Подключить SMTP-сервис (например, Mailtrap) для email-подтверждения.
  - Ввести поле `is_active` в модель пользователя и проверку при авторизации.
  - Унифицировать `get_current_user`, исправить ошибку в `creat_access_token`.
- 

### ☑ 3. Post Service (создание, получение постов)

#### Выполнено:

- Отдельный FastAPI-сервис `posts_service`.
- Реализована модель `Post`, выделена база данных.
- Сервис работает через HTTP и взаимодействует с подписками.

#### 🕒 Недостатки:

- Используется синхронный SQLAlchemy, в отличие от других сервисов.
- Лайки и комментарии не реализованы.
- Авторизация есть, но токен не валидируется — нет защиты эндпоинтов.

#### 💡 Рекомендации:

- Перевести на `async SQLAlchemy` для единообразия.
  - Реализовать эндпоинты `/like`, `/comment`.
  - Добавить валидацию токена через `get_current_user`.
- 

#### ☑ 4. Subscription Service (follow/unfollow)

##### Выполнено:

- Подключён к своей БД, Alembic настроен.
- Реализованы `subscribe`, `unsubscribe`, есть модель `Subscription`.

##### ⊗ Недостатки:

- Не реализованы просмотр подписчиков/подписок.
- Не реализована лента (`/feed`) — она закомментирована и не работает.

##### 💡 Рекомендации:

- Добавить:
    - `/subscriptions/following/{user_id}`
    - `/subscriptions/followers/{user_id}`
    - `/feed/{user_id}` — делает запрос к `posts_service`.
- 

#### ☑ 5. Интеграция через RabbitMQ и FastStream

##### Выполнено:

- RabbitMQ прописан в `docker-compose`.
- В зависимостях всех сервисов есть `faststream`.

##### ⊗ Недостатки:

- Нет кода публикации и подписки на события (`@broker.subscriber` отсутствует).
- Нет интеграции событий `user.registered`, `post.created`.

##### 💡 Рекомендации:

- В `auth_service`:

```
await broker.publish({"user_id": user.id}, "user.registered")
```

- В `subscription_service` — слушать `user.registered` и сохранять авто-подписки.
  - В `post_service` — публиковать событие `post.created`.
- 

#### ⊗ 6. Интеграционные тесты

##### Частично выполнено:

- Присутствует `conftest.py` с фикстурами, `httpx.AsyncClient` подключён.
- Реализован 1 тест feed — проверка `/feed`, но тесты по подписке, постам и авторизации отсутствуют.

 **Рекомендации:**

- Добавить тесты:
  - Регистрация и логин пользователей.
  - Создание поста.
  - Подписка на пользователя.
  - Проверка, что пост попадает в `/feed`.

---

**100** Общая оценка выполнения спринта:

| Критерий                        | Статус                                                |
|---------------------------------|-------------------------------------------------------|
| Микросервисная структура        | <input checked="" type="checkbox"/> Выполнено         |
| Auth Service (регистрация, JWT) | <input type="radio"/> Частично                        |
| Post Service (CRUD, валидация)  | <input type="radio"/> Частично                        |
| Subscription (follow/feed)      | <input type="radio"/> Частично                        |
| RabbitMQ + FastStream           | <input type="radio"/> Подключено, но не интегрировано |
| Интеграционные тесты            | <input type="radio"/> Почти отсутствуют               |

---

 **Общие рекомендации на будущее:**

1. **Добавить документацию API** — использовать Swagger UI FastAPI.
2. **Покрыть ключевые сценарии тестами** — авторизация, постинг, подписка, лента.
3. **Реализовать событийную модель** — через FastStream и RabbitMQ.
4. **Единый `get_current_user` для всех сервисов** — вынести в shared-библиотеку или отдельный auth-gateway.
5. **Подключить Nginx gateway** — для маршрутизации запросов и защиты сервисов.
6. **Добавить healthchecks** для контейнеров (RabbitMQ, сервисы).
7. **Упорядочить зависимости** (`requirements.txt`, убрать неиспользуемые пакеты).